

have already studied recursion in our study of the C programming language. Recall that when writing recursive methods, you must have an **if** statement somewhere to force the method to return without the recursive call being executed. Otherwise the method will never return.

4.11 ACCESS SPECIFIERS

Encapsulation links the data with the code that manipulates the data. Encapsulation also provides another important attribute viz. access control. By introducing access control you can control what parts of the program can access the members of a class and thus prevent misuse. In this section we shall study access control as it applies to classes. An access specifier determines how a member can be accessed. The access specifiers in Java are : **public**, **private** and **protected**. **protected** applies only when inheritance is involved. Java also defines a default access level.

When a member of a class has the specifier **public** associated with it, then such a member can be accessed by any other code in your program. When a class member is specified as **private**, then that member can be accessed by the members of that class alone. Therefore the **main()** method is always declared as **public**, since it is called by code that is outside the program i.e. the Java run time system. When no access specifier is used, then by default the member of a class is **public** within its own package, but cannot be accessed outside the package. A **package** is a grouping of classes. We shall study more about packages later.

An access specifier precedes the rest of a member's type specification. eg.

```
private int i;
```

```
public double a;
```

are both examples of declaring variables with their access specifiers. This means that the access specifier begins the declaration statement of the member.

4.12 STATIC KEYWORD

In some situations, it may be necessary to define a class member which will be used independent of any object of the class. To create such a member we make use of the keyword **static**. When a member is declared **static**, it can be accessed before any objects of its class are created and without reference to any object. Both methods and variables can be declared **static**. Thus you can understand why the **main()** method is declared **static**. **main()** is declared **static** because it must be called before any object exists.

Instance variables declared as **static** are essentially global variables. When objects of its class are declared, no copy of static variable is made. Instead all instances of the class share the same static variable.

Methods declared **static** have the following restrictions :

- they can only call other static methods

- they must access only static data, it is illegal to refer to any instance variable inside of a static method.

- they cannot refer to **this** or **super** in any way.

Outside of the class in which they are created static methods and variables can be used independantly, without use of any object. The general form is :

`classname-method();`

where classname is the name of the class in which the static method is declared.

4.9 to 4.11 Check Your Progress.

1. Write True or False.

- a) Java supports recursion.
- b) A method can return data of the class type that you created.

2. Match the following.

Column A

Column B

- | | |
|--------------|--|
| a) private | i) applies when inheritance is involved |
| b) public | ii) can be accessed by the members of that class alone |
| c) protected | iii) can be accessed by any other code in your program |

4.13 FINAL KEYWORD

When a variable is declared **final**, its contents cannot be modified. You must initialise a **final** variable at the time of its declaration. Usage of **final** is similar to **const** in C/C++. eg.

```
final PI = 3.141
```

```
final MAX = 100;
```

It is a common coding practice to choose all uppercase identifiers for variables declared as **final**.

A note about Arrays : In Java, arrays are implemented as objects. Arrays have a special attribute which is found in its **length** instance variable. The size of the array i.e. the number of elements that the array can hold, is found in the **length** instance variable. All arrays have this variable and it will always hold the size of the array. Remember that the value of length has nothing to do with the number of elements that are actually in use. It only reflects the number of elements that the array is designed to hold. The following program demonstrates how to determine the size of the array using the **length** instance variable.

```
class Length {  
    public static void main(String args[]) {  
        int a1 = new int[10];  
        int a2 = {2, 4, 6, 8, 10, 12};  
        System.out.println("length of a1 is " + a1.length);  
    }  
}
```

```

        System.out.println("length of a2 is " + a2.length);
    }
}

```

The output of the program will be :

length of a1 is 10

length of a 2 is 6

4.12 & 4.13 Check Your Progress.

1. Answer the following.

a) Why is the main() method declared static?

b) What happens when a variable is declared final?

c) What does the length instance variable tell about an array?

4.14 NESTED AND INNER CLASSES

It is possible to define a class within another class; such classes are known as nested classes. If class B is defined within class A, then B is known to A, but not known outside A. A nested class can have access to the members of the class in which it is nested (including private members). However, the enclosing class does not have access to the members of the nested class.

There are two types of nested classes : **static** and **non static**. Static classes have the **static** modifier applied to them, which implies that they must access members of its enclosing class through an object and not directly. Due to this limitation, static nested classes are used rarely.

The most important type of nested class is the **inner** class which is a non static nested class. It has access to all the members of its outer class and can refer to them directly. Thus an **inner** class is fully within the scope of its enclosing class.

Let us study the following illustration of **inner** class.

```

Class Outer {
    int outer_i = 100;
    void test() {

```

```

        Inner inner = new Inner();
        inner.display();
    }
    class Inner {
        void display() {
            System.out.println("Display member variable of
outer : " + outer_i);
        }
    }
}
class InnerOuter {
    public static void main(String args[]) {
        Outer outer = new Outer();
        outer.test();
    }
}

```

The output of the above program is :

Display member variable of outer : 100

You can see from the above example, that the inner class named **Inner** is defined within the scope of the outer class named **Outer**. **Inner** directly access the variable `outer_x`. The **display()** method is defined in the inner class and the outer class creates and instance of the inner class to access the method **display()** as **inner.display()**. Remember that no code outside of class **Outer** can access the class **Inner**. Also remember that an inner class can be defined within any block scope. It is not necessary that an inner class be enclosed within the scope of a class only.

4.14 Check Your Progress.

1. Write True or False

- a) A nested class cannot have access to the members of the class in which it is nested.
- b) An inner class can be defined within any block scope.

4.15 INHERITANCE

4.15.1 Inheritance a Class

We have already studied that using inheritance we can create a general class that defines traits common to a set of related items. This class can then be inherited by other more specific classes. Each of these classes adds those features that are unique to it. A class that is inherited is called the **superclass**.

The class which inherits another class is called a **subclass**. Thus a subclass is a specialised version of a superclass, which inherits all the instance variables and methods defined by the superclass, at the same time adding its own unique elements.

In order to inherit a class we make use of the keyword **extends**. The general form of extending a class is :

```
class subclass-name extends superclass-name {  
    body of the class  
}
```

The following example will demonstrate how to inherit a class.

```
Class A {  
    int x, y;  
    void display() {  
        System.out.println("x and y " + x + y);  
    }  
}  
  
class B extends A {  
    int z;  
    void showz() {  
        System.out.println("z = " + z);  
    }  
    void add() {  
        System.out.println("x+y+z= " + (x + y + z));  
    }  
}  
  
class Inherits {  
    public static void main(String args[]) {  
        A a = new A();  
        B b = new B();  
        a.x = 10;  
        a.y = 5;  
        System.out.println("Contents of Superclass A :");  
        a.display();    // invoke display on object a  
        b.x = 20;        //Subclass can access public  
                        members of superclass  
        b.y = 30;  
        b.z = 10;
```

```

        System.out.println("Contents of Subclass ");
        b.display();           // invoke the display
                               method on object b

        b.showz();
        System.out.println("Call add method from
        Subclass :");
        b.add();               // add method of subclass
    }
}

```

The output of the program will be :

Contents of Superclass A :

10 5

Contents of Subclass

x and y : 20 30

z = 10

x+y+z= 60

Here you can see that the subclass B includes all the members of its superclass A. Therefore it can access the variables x and y. Note that class A though a superclass of B, is completely independent stand alone class.

Remember that you can only specify one superclass for any subclass that you create. Inheritance of multiple superclasses is not possible in Java. (This differs from C++). However, a subclass can be a superclass for another subclass. Thus you can create a hierarchy of inheritance in which a subclass becomes a superclass of another superclass. No class can be a superclass of itself. A superclass can be used to create any number of subclasses. Also remember that a member which has been declared **private** is not accessible by any code outside its class, including the subclasses.

4.15.2 Super

Whenever a subclass needs to refer to its immediate superclass, it can do so by using the keyword **super**. **super** has two general forms. The first one calls the constructor of the superclass. The second one is used to access a member of the superclass that has been hidden by a member of a subclass.

In the first form of **super**, a subclass can call a constructor method defined by its superclass by using the following form :

```
super(parameter-list);
```

The *parameter-list* specifies any parameters needed by the constructor in the superclass. Remember that **super()** must always be the first statement executed inside a subclass' constructor. The following example will illustrate this use of **super**

```

class A {
    int i;
    int j;
    A(int a, int b)
    {
        i = a;
        j = b;
    }
    ...
    ...
}

class B extends A {
    int k;
    B(int p, int q, int r) {
        super(p,q);
        k = r;
    }
    ...
}

```

Here the constructor for subclass B is called with parameters p, q, r. We make use of **super()** with parameters p and q which initialise i and j. The class A no longer initialises these values itself. Remember that constructors can be overloaded, therefore **super()** can be called using any form defined by the superclass. The one that matches the arguments in number and type will be executed.

An important thing to note is that **super()** always refers to the superclass immediately above the calling class. The general form of **super** in the second method is:

super.member

member can be either a method or an instance variable. This form of **super** is applicable when members of the subclass hide the members of the superclass which have the same name in both classes. **super** allows access to the member of the superclass which has the same name as a member of the subclass.

eg. if a variable i is defined in both the superclass and the subclass, then using **super.i** in the subclass, you can refer to the variable i of the superclass.

4.15.3 Multilevel Hierarchy

You can build hierarchies which contain as many levels of

inheritance as you like. eg. if you have three classes A, B and C, then C can be a subclass of B, where B is a subclass of A. In this case, C inherits all the properties of A and B. Remember that all classes can be written in separate files and compiled separately.

In a class hierarchy, constructors are called in their order of derivation, from superclass to subclass. Also remember that **super()** must be the first statement executed in the subclass constructor.

4.15.1 to 4.15.3 Check Your Progress.

1. Fill in the blanks.

- a) A class that is inherited is called the
- b) We make use of the keyword to inherit a class.
- c) In a class constructors are called in their order of derivation.

2. What are the two general forms of super?

.....
.....

4.15.4 Method Overriding

It may so happen that a method in a subclass has the same name and type as that of the superclass. In this situation, the method in the subclass is said to override the method in the superclass and the method defined by the superclass will be hidden. Overridden methods in Java are similar to virtual functions in C++. Let us illustrate method overriding with the following example :

```
class A {  
    int i, j;  
    A(int a, int b) {  
        i = a;  
        j = b;  
    }  
    void display() {  
        System.out.println( "i = " + i + "j = " + j);  
    }  
}  
class B extends A {  
    int k;  
    B(int a, int b, int c) {  
        super(a,b);  
        k = c;  
    }  
    void display() {
```

```

        System.out.println("k = " + k);
    }
}
class Override {
    public static void main(String args[ ]) {
        B b = new B(4, 10, 12);
        b.display();
    }
}

```

The output of this program will be :

k = 12

Here, when you invoke the method `display()` on an object of type B, the version of `display()` which has been defined in B will be used. This means the version of B will override the version declared in A.

If you wish to access the method of the superclass, you can do so by using the keyword **super**. Modify the above example as shown where you will precede the `display()` method of B with the word **super**.

```

class B extends A {
    int k;
    B(int a, int b, int c) {
        super(a, b);
        k = c;
    }
    void display() {
        super.display() // call display of A
        System.out.println("k = " + k);
    }
}

```

The output of the modified program will be :

i = 4 j = 10

k = 12

`super.display()` calls the superclass version of `display()`.

Remember that method overriding occurs only when the names and the type signatures of the two methods are identical. If they are not then the methods are simply overloaded.

4.15.5 Dynamic Method Dispatch

This is the mechanism by which a call to an overridden function is

resolved at run time rather than compile time. This is how Java implements run time polymorphism. When a method that is overridden is called through a superclass reference, Java determines which version of that method to execute, based upon the object being referred to at the time the call occurs. Thus, this determination is made at run time. Thus, it is the type of the object being referred to and not the type of the reference variable, which determines the particular version of the method to be executed.

4.15.4 to 4.15.5 Check Your Progress.

1. Answer the following.

- a) When does a method in the subclass override a method in the superclass?

- b) What is meant by dynamic method dispatch?

4.15.6 Using Abstract Classes

Sometimes it is necessary to create a superclass that only defines a generalised form. This generalised form will be shared by all its subclasses and each subclass will fill in its own details. Such a class determines the nature of the methods that the subclass must implement. You can require that a certain method be overridden by subclasses by specifying the **abstract** modifier. The general form to declare an **abstract** method is :

```
abstract type name(parameter-list);
```

Note that no method body is present. Any class which contains one or more **abstract** methods must also be declared **abstract**. There can be no object of an **abstract** class. This means that an abstract class cannot be instantiated with the **new** operator. You cannot declare **abstract** constructors or **abstract static** methods. Any subclass of an abstract class must either implement all of the abstract methods of the superclass, or itself be declared **abstract**. A class is declared **abstract** by writing the **abstract** keyword before the class keyword at the beginning of the class declaration.

The following example will illustrate :

```
abstract class A {  
  
    abstract void meth();  
    void meth1() {  
        System.out.println("Method in abstract class A");  
    }  
}
```

```

class B extends A {
    void meth() {
        System.out.println("Implement the abstract method of
superclass A");
    }
}

class Abstracts {
    public static void main(String args[]) {
        B b = new B();
        b.meth ();
        b.meth1();
    }
}

```

The output of the program would be :

Implement the abstract method of superclass A

Method in abstract class A

Remember that you cannot create an instance of class A. The method **meth()** which is **abstract** in class A is provided a body in the subclass B.

4.15.7 final

We have already seen one use of **final** to create a named constant. Let us study two more uses of **final** in this section.

Using final to prevent overriding : To disallow a method from being overridden, specify the **final** modifier at the start of the declaration of the method. This means that methods declared as **final** cannot be overridden. Once a method has been declared **final** in a class, its subclass cannot override it. There can be no method with the same name and type in the subclass. A compile time error will occur in such situation.

Using final to prevent inheritance : If you want to prevent a class from being inherited you precede the class declaration with the word **final**. When a class is declared **final**, all of its methods are also implicitly declared **final**. It is illegal to declare a class as both **abstract** and **final**, because an abstract class is incomplete by itself and requires the subclass to provide implementation to its methods.

4.15.6 to 4.15.7 Check Your Progress.

1. Write True or False.

- An abstract class can be instantiated with the new operator.
- A class can be declared as both abstract and final.

2. List two uses of final.

4.16 THE OBJECT CLASS

Object class is a special class in Java. All the other classes are subclasses of **Object**. **Object** is the superclass of all the classes. This means that a reference variable of type **Object** can refer to an object of any other class. Also remember that since arrays are implemented as classes, a variable of type **Object** can also refer to any array.

The **Object** class defines the following methods :

Method	Purpose
Object clone()	Creates a new object that is the same as the object being cloned
boolean Equals(Object object)	Determines whether one object is equal to another
void finalize()	called before an unusual object is recycled
Class getClass()	Obtains the class of an object at run time
int hashCode()	Returns the hash code associated with the invoking object
void notify()	Resumes execution of a thread waiting on the invoking object
void notifyAll()	Resumes execution of all threads waiting on the invoking object
String toString()	Returns a string that describes the object
void wait()	Waits on another thread of execution

Among these, the methods **getClass()**, **notify()**, **notifyAll()** are declared as **final**. Other methods may be overridden. We shall see all these methods in detail as we progress through the study notes.

4.16 Check Your Progress.

1. Fill in the blanks.

- The class is the superclass of all the classes.
- The getClass() method is declared as in object.

4.17 SUMMARY

A class is a logical construct which defines the shape and nature of an object.

The variables defined within a class are called instance variables. The code is contained within methods. The variables and the methods within a class are called as members of the class. The new operator dynamically allocates memory for an object and returns a reference to it.

Constructors : Java allows objects to initialise themselves when they are created. This automatic initialisation is performed through the use of a constructor. A constructor has the same name as the name of the class in which it resides. Constructors do not have a return type.

Garbage collection : Java handles deallocation automatically. The technique which accomplishes this task is called garbage collection.

finalize() method : You can define a finalize() method to specify those actions that must be performed before an object is destroyed.

The call by value method copies the value of an argument into the formal parameter of the subroutine. In the call by reference method, a reference to an argument is passed to the parameter.

Class declared with in a class called nested class.

There are two types of nested classes : static and non static.

4.18 CHECK YOUR PROGRESS - ANSWERS

4.1 & 4.2

1. a) True
b) False
c) True
d) False
2. a) new
b) class, instance

4.3

1. a) void
b) argument
c) parameter

4.4 & 4.5

1. a) False
b) True
c) False
d) True

4.6

1. a) False

- b) True
- c) False

4.7 & 4.8

1. a) True
 - b) False
 - c) False

4.9 to 4.11

1. a) True
 - b) True
2. a) ii)
 - b) iii)
 - c) (i)

4.12 & 4.13

1. a) main() method is declared as static because when a member is declared static, it can be accessed before any objects of its class are created and main() has to be called before any object exists.
 - b) When a variable is declared final its contents cannot be modified.
 - c) The size of the array i.e. the number of elements that the array can hold is found in the length instance variable of an array.

4.14

1. a) False
 - b) True

4.15.1 to 4.15.3

1. a) superclass
 - b) extends
 - c) hierarchy
2. The two general forms of super are :
 - `super(parameter-list);`
 - `super.member`

4.15.4 & 4.15.5

1. a) When a method in a subclass has the same name and type as that of the superclass the method in the subclass overrides the method in the superclass and the method defined by the superclass will be hidden.
 - b) Dynamic method dispatch is the mechanism by which Java implements run time polymorphism whereby a call to an overridden function is resolved at run time rather than compile time.

4.15.6 & 4.15.7

1. a) False
b) False
2. Two uses of final : to prevent overriding, to prevent inheritance.

4.16

1. a) object
b) final

4.19 QUESTIONS FOR SELF - STUDY

1. What is a class? Describe the general form of declaring a class.
2. What are methods? What is the general form of declaring a method?
3. **Write short notes on :**
 - a) Constructors
 - b) Parameterised methods
 - c) Call by value and call by reference
 - d) Access specifiers in Java
 - e) The final keyword
 - f) Method overriding
 - g) Abstract class
4. What is overloading?
5. What do you understand by garbage collection? Describe in brief the finalize() method.
6. Describe the use of the static keyword in Java.
7. Write in brief about nested and inner classes.
8. Describe what is meant by inheritance. What do you understand by multilevel hierarchy?
9. What is meant by dynamic method dispatch?
10. What is the Object class? Write any two methods defined in the Object class.

4.20 SUGGESTED READINGS

1. www.java.com
2. www.freejavaguide.com
3. www.java-made-easy.com
4. www.tutorialspoint.com
5. www.roseindia.net



[illegible]

Notes

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Chapter : 5

Packages and Interfaces

5.0	Objectives
5.1	Introduction
5.2	Packages
5.2.1	Classpath
5.2.2	Definition Packages
5.2.3	Catagories of Visibility for class members with reference to packages
5.2.4	Importing Packages
5.3	Interfaces
5.3.1	Declaring an Interface
5.3.2	Implementing an interface
5.3.3	Variables in Interfaces
5.4	Summary
5.5	Check Your Progress - Answers
5.6	Questions for Self - Study
5.7	Suggested Readings

5.0 OBJECTIVES

Dear Students,

After studying this chapter you will able to :

- + Describe what is meant by a package
- + Explain how to define a package
- + State the way of importing packages in programs
- + Describe what is an interface
- + Explain how to define and implement an interface

5.1 INTRODUCTION

Packages are containers for classes which are used to keep the class name space compartmentalised. They are stored in a hierarchical manner and are explicitly imported into new class definitions. **Interfaces** can be used to specify a set of methods which can be implemented by one or more classes. The interface, itself does not define any implementation. An interface is similar to an abstract class, however a class can implement more than one interface. On the other hand, a class can only inherit a single superclass. Packages and interfaces are two of the basic components of a Java program.

In general a Java source file contains the following four internal parts :

- A single package statement (optional)

- Any number of import statements(optional)
- A single public class declaration (required)
- Any number of classes private to the package (optional)

Till now, we have only used the single **public** class declaration in all our programs. We shall see the remaining in this section.

5.2 PACKAGES

5.2.1 CLASSPATH

Before studying packages it is important to understand **CLASSPATH**. The specific location that the Java compiler will consider as the root of any package is controlled by **CLASSPATH**. Till this time, we have been storing all our classes in the same, unnamed default package. This allows you to compile the source code and run the Java interpreter on the compiled program by giving the name of the class on the command line. This works because the default current working directory is usually in the **CLASSPATH** environmental variable. This variable is defined for the Java run time system by default.

Suppose you create a class **MyPack** in a package called **pack**. This means you have to create a directory called **pack** and put **MyPack.java** into this directory. **pack** should now be made the current directory to compile the program. The class file **MyPack.class** which will be created after compilation will also be stored in the **pack** directory. Now when you try to run **MyPack**, the Java interpreter reports an error. This is because the class is now stored in a package called **pack**. Therefore you cannot refer to it only as **MyPack**. You should now refer to the class by enumerating its package hierarchy and separating the package by dots. Thus the class will now be called **MyPack.pack**. Now if you try using **MyPack.pack**, you will still receive an error message. This is because **CLASSPATH** sets the top of the class hierarchy. So there is no directory called **pack**, in the current working directory since you are working in **pack** itself.

To overcome this problem you have two alternatives :

- change directories up one level and type `java MyPack.pack`
- add the top of your development class hierarchy to the **CLASSPATH** environment variable. Then you will be able to use `java MyPack.pack` from any directory and Java is able to find the correct **.class** file. eg. if you are working on source code in a directory called `c:\student` then you should set your **CLASSPATH** as :

`.,C:\student;C:\java\classes`

5.2.2 Defining Packages

You can define classes inside a package that are not accessible to the code outside the package. You can also define class members that are only exposed to other members of the same package. Thus the classes can have knowledge of

each other, but it is not exposed outside the package.

To create a package, you should include a package command as the first statement in a Java source file. Any classes defined within that file will belong to the specified package.

The general form for the package statement is :

```
package pkg;
```

where *pkg* is the name of the package. eg.

```
package pack
```

will create a package named pack.

Java uses the file system directories to store packages. Therefore any **.class** files for any classes declared to be a part of **pack**, must be stored in a directory called **pack**. The directory name must match the package name.

More than one file can be included the same package statement. You can also create a hierarchy of packages. The general form of a multilevel package statement is :

```
package pkg1[.pkg2 [pkg3]];
```

remember that the package hierarchy should be reflected in the file system of your Java development system. eg. package **java.awt.image**;

should be stored in **java\awt\image** on your Windows file system.

Let us create a small package with the following example :

```
package Pack;
```

```
class Student {
```

```
    String name;
```

```
    int roll;
```

```
    Student(String n, int r) {
```

```
        name = n;
```

```
        roll = r;
```

```
    }
```

```
    void display() {
```

```
        System.out.println("Name : " + name + " Roll No. : " + roll);
```

```
    }
```

```
}
```

```
class Studrec {
```

```
    public static void main(String args[]) {
```

```
        Student S[] = new Student[3];
```

```
        S[0] = new Student("Vikram ", 20);
```

```
        S[1] = new Student("Jerry ",10);
```

```
        S[2] = new Student("Vishnu ",15);
```

```

        for(int i = 0; i<3; i++)
            S[i].display();
    }
}

```

Save this file as **Studrec.java** and put in a directory called **Pack**. When you compile this file, make sure that the resulting **.class** file has also been stored in Pack. Then execute **Studrec** at the command line as follows :

```
java Pack.Studrec
```

Either set the **CLASSPATH** environment as described previously or go into the directory above Pack to execute the program. Remember that since **Studrec** is now a part of the package **Pack**, it cannot be executed by itself i.e. a *command line*

```
java Studrec
cannot be used.
```

5.2.1 & 5.2.2 Check Your Progress.

1. Write True or False.

- A class can implement more than one interface.
- You should include a package command as the first statement in a Java source file.
- You can define classes inside a package that are not accessible to the code outside the package.

5.2.3 CATAGORIES OF VISIBILITY FOR CLASS MEMBERS WITH REFERENCE TO PACKAGES

Packages act as containers for classes and other subordinate packages. Java addresses four catagories of visibility for class members with reference to packages :

- subclasses in the same package
- non subclasses in the same package
- subclasses in different packages

	Private	No modifier	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package				
subclass	No	Yes	Yes	Yes
Same package				
non subclass	No	Yes	Yes	Yes
Different package				
subclass	No	No	Yes	Yes
Different package				
non subclass	No	No	No	Yes

- Classes that are neither in the same package nor subclass.

The following table summarises the class member access

Anything declared **public** can be accessed from anywhere. Anything declared **private** is not accessible outside of its class. When a member does not have any specific access specification, it is visible to all subclasses as well as to other classes in the same package. This is the default access. If you want to allow an element to be seen outside your current package but only to classes that are the direct subclasses of your class, you have to declare that element as **protected**.

5.2.4 Importing packages

All of the built in Java classes are stored in packages. In order to make visible certain classes or entire packages, Java makes use of the **import** statement. Once imported, a class can be referred to directly using only its name. In a Java source file, the **import** statements occur immediately following the package statement (if it exists) and before any class definitions. The general form of the import statement is :

```
import pkg1[.pkg2].(classname|*);
```

pkg1 is the name of the top level package, *pkg2* is the name of the subordinate package and is separated by a dot. Practically there is no limit to the depth of a package hierarchy. Then you can specify either an explicit *classname* or a star (*). A * indicates that the Java compiler should import the entire package. eg.

```
import java.util.Date;
import java.io.*;
```

All the standard Java classes included with Java are stored in a package called **java**. The basic language functions are stored in a package called **java.lang**. **java.lang** is implicitly imported by the compiler for all the programs because it provides a lot of functionality. This is equivalent to importing it as :

```
import java.lang.*;
```

If a class with the same name exists in two different packages which you import using the * form, then compiler does not issue any warning unless you try to use one of the classes. In that case a compile time error occurs and you have to explicitly name the class and its package name.

Wherever you use a class name you can use its fully qualified name which will include its full package hierarchy. eg.

```
import java.util.*;
class Example extends Date {
}
can be written without import as :
class Example extends java.util.Date {
}
```

Only those items within the package declared as **public** will be available to non subclasses in the importing code.

5.2.3 & 5.2.4 Check Your Progress.

1. Fill in the blanks.

- a) The basic language functions in Java are stored in a package called
- b) To make visible certain classes or entire packages, Java provides the statement.
- c) To allow an element to be seen outside the current package, but only to classes that are the direct subclasses of your class, that element should be declared as

5.3 INTERFACES

5.3.1 Declaring an Interface

Using an interface you can specify what a class must do, but not how it does it. You can fully abstract a class' interface from its implementation. Interfaces do not have instance variables and their methods do not have any bodies. Once an interface is defined, any number of classes can implement an interface. Also one class can implement any number of interfaces. In order to implement an interface, a class must create a complete set of methods defined by the interface, but each class is free to implement its own details.

Interfaces are designed to support dynamic method resolution at run time. Usually, for a method to be called from one class to another class, both the classes need to be present at compile time. Since interfaces are in a different hierarchy from classes, it is possible for classes that are unrelated in terms of the class hierarchy to implement the same interface.

The general form of an interface is :

```
access interface name {  
    return-type method-name1(parameter-list);  
    return-type method-name2(parameter-list);  
    return-type method-name2(parameter-list);  
    .....  
    return-type method-nameN(parameter-list);  
    type final-varnameN = value;  
}
```

In this form, access is either **public** or not used. The default access applies when no specifier is used. It implies that the interface is only available to other members of the package in which it is declared. When the interface is declared **public**, the interface can be used by another code. *name* is the name of the interface which is a valid identifier. The methods in an interface are abstract methods. Each class which includes an interface must implement all the methods.

Variables can also be declared inside interface declarations. They are implicitly **final** and **static**. This means they cannot be changed by the classes which implement the interface. The variables must be initialised with constant values. If an interface is declared **public**, then all the methods and variables are implicitly declared **public**.

Let us see an example of interface declaration :

```
interface A {  
    void meth1(int i);  
}
```

5.3.1 Check Your Progress.

1. Fill in the blanks.

- a) Variables declared in interfaces are implicitly and
- b) The access applies when no specifier is used for an interface.
- c) Interfaces do not have variables.

5.3.2 Implementing an interface

To implement an interface include the **implements** clause in the class definition and then create the methods that are defined by the interface. The general form of a class which includes the implements clause is :

```
access class classname [extends superclass]  
    [implements interface [,interface....]] {  
    class body  
}
```

access is either **public** or not used. If a class implements more than one interface they should be separated by commas. The methods that implement an interface must be declared **public**. Also note that classes which implement interfaces, can have additional members of their own also.

The following example will demonstrate how to implement the above declared interface :

```
class One implements A {  
    public void meth1(int i) {  
        System.out.println("Implement interface A in class One");  
        System.out.println("i is :" + i);  
    }  
}
```

You can also declare variables as object references that use an interface rather than a class type. Any instance of any class that implements the declared interface can be stored in such a variable. When a method is called through any such references, the correct version based upon the actual instance of the interface

being referred to will be called. The method to be executed is looked up dynamically at run time. This allows classes to be created later than the code which calls methods on them.

Let us call the method **meth1()** in the above example with the help of an interface reference variable as shown :

```
class Iface {  
    public static void main(String args[]) {  
        A Ainter = new One();  
        Ainter.meth1(10);  
    }  
}
```

In this example, the variable Ainter is of interface type A, but it is assigned an instance of class One. Ainter can access meth1(), but it cannot access any other members of the class One because the interface reference variable has only knowledge of the methods declared by that interface declaration. It cannot be used to access any other methods of the class.

If a class includes an interface but does not fully implement the methods defined by that interface, then the class must be declared **abstract**. eg

```
abstract class Two implements A {  
    int p, q;  
    void display() {  
        System.out.println("p and q" + p + q);  
    }  
}
```

This class does not implement meth1() of the interface A therefore is declared **abstract**.

Interfaces can be extended i.e. one interface can inherit another by the use of the keyword **extends**. When a class implements an interface which inherits another interface, then it must provide implementations for all the methods defined within the interface inheritance chain.

5.3.3 Variables in Interfaces

You can use interfaces to import shared constants into multiple classes by simply declaring an interface that contains variables which are initialised to the desired values. When the interface is included in a class all those variables names will be in scope as **constants**. (This is similar to a header file in C/C++ which contains a number of **#define** constants or **const** declarations).

5.4 SUMMARY

Packages are containers for classes which are used to keep the class name space compartmentalized. Packages act as containers for classes and other subordinate packages.

Interfaces can be used to specify a set of methods which can be implemented by one or more classes.

5.5 CHECK YOUR PROGRESS - ANSWERS

5.2.1 & 5.2.2

1. a) True
b) True
c) True

5.2.3 & 5.2.4

1. a) java.lang
b) import
c) protected

5.3.1

1. a) static
b) default
c) instance

5.3.2 & 5.3.3

1. a) False
b) True
2. a) implements
b) abstract

5.6 QUESTIONS FOR SELF - STUDY

1. What is meant by packages and interfaces?
2. Explain with example what is CLASSPATH.
3. Explain the general form of declaring a package.
4. What do you understand by public, private and protected access, with relation to packages?
5. Write notes on :
 - a) Declaring an Interface
 - b) Implementing an Interface

5.7 SUGGESTED READINGS

1. www.java.com
2. www.freejavaguide.com
3. www.java-made-easy.com
4. www.tutorialspoint.com
5. www.roseindia.net

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Chapter : 6

Exception Handling

6.0	Objectives
6.1	Introduction
6.2	Exception Handling
6.3	Exception Types
6.4	Using try and catch
6.5	Multiple catch clauses
6.6	Nested try statements
6.7	throw
6.8	throws
6.9	finally
6.10	Built in exceptions in Java
6.11	Summary
6.12	Check Your Progress - Answers
6.13	Questions for Self - Study
6.14	Suggested Readings

6.0 OBJECTIVES

Dear Students,

After studying this chapter you will be able to

- discuss what is an exception
- explain exception handling with the keywords try, catch, throw, throws and finally
- discuss the exception types
- explain how to use the try and catch clauses, multiple catch and nested try statements
- discuss the use of throw, throws and finally

6.1 INTRODUCTION

An exception in Java is an object that describes an exceptional i.e. error condition which has occurred in a piece of code. When such an exception condition arises, an object representing that exception is created and thrown in the method that caused the error. The method may handle the exception on its own or may pass it on. In any case, at some point the exception gets caught and is processed.

Exceptions can be generated by the Java run time system, or they can be manually generated by your code. Exceptions thrown by Java relate to fundamental errors which violate the rules of the Java language or the constraints of the Java execution environment. Manually generated exceptions can be used to report error conditions to the caller of a method.

6.2 EXCEPTION HANDLING

Exception handling in Java is managed by five keywords : **try**, **catch**, **throw**, **throws**, and **finally**. A **try** block contains the program statements which you want to monitor for exceptions. If an exception occurs within the **try** block, then it is thrown. Using **catch**, your code can **catch** this exception and handle it in the appropriate manner. The exceptions generated by the system are automatically thrown by the Java run time system. The keyword **throw** is used to manually throw an exception. Any exception that is thrown out of a method is specified by the **throws** clause. Any code that must absolutely be executed before a method returns is put in a **finally** block.

The general form of an exception handling block is :

```
try {  
    // try block to monitor for errors  
}  
catch(ExceptionType1 exOb) {  
    // exception handler for ExceptionType1  
}  
catch(ExceptionType2 exOb) {  
    // exception handler for ExceptionType2  
}  
// ....  
finally {  
    // block of code to be executed before try block ends  
}
```

ExceptionType is the type of exception that has occurred.

6.3 EXCEPTION TYPES

All exceptions are subclasses of the built in class **Throwable**. **Throwable** class is at the top of the exception hierarchy. Immediately below **Throwable** are two subclasses. These subclasses partition the exceptions in two distinct branches. One branch is **Exception**. This class is used for exceptional conditions that user programs should catch. This is also the class that you will subclass to create your own custom exception types. An important subclass of **Exception** is called

Runtime Exception. Exceptions of this type are automatically defined for the programs that you write and include things such as division by zero, invalid array indexing etc.

The other branch is topped by **Error**. This defines the exceptions that are not expected to be caught under normal circumstances by your program. Exceptions of the type **Error** are used by the Java run time system to indicate errors which relate to the run time environment itself. eg. stack overflow. In this chapter we shall not deal with exceptions of type **Error**.

6.1 to 6.3 Check Your Progress.

1. Fill in the blanks.

- a) All exceptions are subclasses of the class
- b) Immediately below Throwable are two subclasses and
- c) The keyword is used to manually throw an exception.
- d) Any code that must absolutely be executed before a method returns is put in a block.

2. Answer the following.

- a) What is an exception?

- b) What do exceptions thrown by Java relate to?

6.4 USING TRY AND CATCH

The default exception handler is provided by the Java run time system due to which exceptions get caught. However, you will usually want to handle an exception yourself. Handling exception by yourself will allow you to fix the error and at the same time prevent automatic program termination (which happens with the default exception handler).

In order to monitor a code for exceptions, enclose it in a **try** block and immediately following the **try** block, include a **catch** clause which specifies the exception type you wish to catch. The following example will illustrate :

```
class Excep {  
    public static void main(String args[ ]) {  
        int a, b;  
        try {
```

```

        b = 0;
        a = 10/b;
    }
    catch(ArithmeticException e) {
        System.out.println("Division by zero error");
    }

    System.out.println("After executing catch
clause");
}
}

```

The output of the program will be :

Division by zero error

After executing catch clause

Remember that once an exception is thrown, control transfers out of the **try** block into the **catch** block. Once the **catch** clause is executed control continues with the next line in the program following the entire **try-catch**. Note that the scope of the **catch** clause is restricted to those statements specified by the immediately preceding **try** statement. The aim of most well constructed **catch** clauses should be to resolve the exceptional condition and then continue as if the exception never happened.

Displaying the Description of an Exception :

Throwable overrides the **toString()** method defined by **Object** and therefore it returns a string which contains the description of the exception. This description can be displayed in a **println()** statement by passing the exception as an argument. eg. The above example can be modified as :

```

    catch(ArithmeticException e) {
        System.out.println("Exception " + e);
    }

```

When the code is modified as above and a division by zero occurs the following will be displayed :

Exception : java.lang.ArithmeticException / by zero

6.5 MULTIPLE CATCH CLAUSES

In situations where more than one exception could be raised by a single piece of code, you can specify two or more **catch** clauses, where each will catch a different type of exception. When an exception is thrown, each **catch** statement is inspected in order and the first one whose type matches that of the exception is executed. No sooner does one **catch** statement execute, the others are bypassed and the control is transferred to the line following the try/catch block. The following

example will demonstrate :

```
class MultipleCatch {  
    public static void main(String args[]) {  
    try {  
        int a = args.length;  
        System.out.println("a= " + a);  
        int b = 2/a;  
        int c[] = {1};  
        c[10] = 25;  
    }  
    catch(ArithmeticException e) {  
        System.out.println("Division by zero : " + e);  
    }  
    catch(ArrayIndexOutOfBoundsException e) {  
        System.out.println("Array index out of bounds " + e)  
    }  
    System.out.println("After try/catch block");  
    }  
}
```

If this program is started with no command line parameters it will cause the division by zero exception. If you provide a command line argument setting a to something greater than zero, then this exception will not occur. However, it will cause **ArrayOutOfBoundsException** since int array c has length 1 and the program attempts to assign a value to c[10].

The output generated in both cases will be as shown below:

```
C:\> java MultipleCatch
```

```
a = 0
```

```
Divide by 0 : java.lang.ArithmeticException: / by zero
```

```
After try/catch block
```

```
C:\>java MultipleCatch Arg
```

```
a = 1
```

```
Array index out of bounds : java.lang.ArrayIndexOutOfBoundsException : 10
```

```
After try/catch block
```

Remember that when you use multiple **catch** statements the exception subclasses must come before any of their superclasses. This is because a **catch** statement that uses a superclass will catch exceptions of that type plus any of its subclasses. Thus a subclass would never be reached if it came after its superclass. Also, in Java unreachable code is an error.

eg. in a multiple catch clause suppose you first attempt to catch an **Exception** and then **ArithmeticException**, then the **catch** statement which catches the **ArithmeticException** will never reach. This is because **ArithmeticException** is a subclass of **Exception**. Therefore the first catch itself will handle all **Exception**-based errors including **ArithmeticException**. To correct this situation, you should first write a catch to tackle **ArithmeticException** and then **Exception**.

6.6 NESTED TRY STATEMENTS

A **try** statement can be enclosed inside the block of another **try**. Each time a **try** statement is entered, the context of its exception is pushed onto the stack. If an inner **try** statement does not have a **catch** for a particular exception, the stack is unwound and the next **try** statement's catch handlers are checked for a match. This continues till a corresponding **catch** is found or till all the nested **try** statements are exhausted. In a situation where no catch statement matches, the Java run-time system handles the exception.

The following example illustrates nested **try**:

```
class Nestedtry {
    public static void main(String args[]) {
        try {
            int a = args.length;
            int b = 2/a;
            System.out.println("a = " + a);
            try {                //nested try block
                if(a != 0) {
                    int c[] = {1};
                    c[10] = a;
                }
            } catch(ArrayIndexOutOfBoundsException e) {
                System.out.println("Array index out of
bounds : " + e);
            }
            } catch(ArithmeticException e) {
                System.out.println("Division by zero : " + e);
            }
        }
    }
}
```

The **try** block to check array index is nested within the try block which checks for division by zero. Thus when the program is executed without any

command line arguments, the exception is generated by the outer try block. If the program is executed with one or more command line arguments then the exception is generated in inner try block. Check the output of this program as an exercise.

Remember that you can enclose a call to a method within a **try** block.

6.4 to 6.6 Check Your Progress.

1. Write True or False.

- a) Once an exception is thrown, control transfers out of the try block into the catch block.
- b) Try statements cannot be nested.
- c) Throwable overrides the toString() method defined by Object.
- d) Exception is subclass of Arithmetic Exception.

6.7 THROW

The **throw** statement allows your program to throw an exception explicitly. The general form of **throw** is :

```
throw ThrowableInstance;
```

ThrowableInstance must be an object of type **Throwable** or a subclass of **Throwable**. Simple types like **int** or **char**, as well as non-Throwable classes like **Strings** cannot be used as exceptions. A **Throwable** object can be obtained in two ways :

- using a parameter into a catch clause
- creating one with the **new** operator.

The flow of execution stops immediately after the **throw** statement, and subsequent statements are not executed. The nearest enclosing **try** block is checked to see whether it has a **catch** statement that matches the type of exception. If a match is found, control is transferred to that statement. If no match is found, the next enclosing **try** statement (if there is one) is inspected and so on. If no match is found, then the default exception handler halts the program. The following example shows the use of **throw** :

```
class ThrowDemo {  
    public static void main(String args[ ]) {  
        try {  
            try {  
                throw new NullPointerException("Use of throw");  
            }  
            catch(NullPointerException e) {  
                System.out.println("Caught Exception thrown by  
throw");  
                throw e;  
            }  
        }  
    }  
}
```

```

    }
}
catch(NullPointerException e) {
    System.out.println("Exception caught : " + e);
}
}
}

```

In this example **new** is used to construct an instance of **NullPointerException**. All of Java's built-in run-time exceptions have two constructors : one with no parameter and one that takes a string parameter. In the second form, the argument specifies a string which describes the exception. This string is displayed when the object is used as an argument to **println()**.

6.8 throws

A **throws** clause lists the types of exceptions that a method might throw. This is necessary for all exceptions, except those of type **Error** or **RuntimeException**, or any of their subclasses. All other exceptions that a method can throw must be declared in the **throws** clause, if they are not a compile time error will occur.

The general form of the *throws* clause is :

```

type method-name(parameter-list) throws exception-list {
    // body of the method
}

```

exception-list is a comma separated list of the exceptions that a method can throw.

Study the following example.

```

class Demothrows {
    static void throwmeth() throws
    IllegalAccessException {
        System.out.println("Inside throwmeth");
        thrownew IllegalAccessException("Demo");
    }
    public static void main(String args[ ]) {
        try {
            throwmeth();
        }
        catch(IllegalAccessException e) {
            System.out.println("Caught
            Exception " + e);
        }
    }
}

```

6.7 & 6.8 Check Your Progress.

1. Fill in the blanks.

- a) The statement allows your program to throw an exception explicitly.
- b) A clause lists the types of exceptions that a method might throw.

2. Write True or False.

- a) Simple types like int or char can be used as exceptions
- b) All of Java's built-in run-time exceptions have two constructors

6.9 finally

finally is used to create a block of code that will be executed after a try/catch block has completed and the before the code following the try/catch block. The **finally** block will be executed whether or not an exception is thrown. If an exception is thrown, the **finally** block will execute even if no **catch** statement matches the exception. The **finally** clause is optional. However, each **try** statement requires at least one **catch** or a **finally** clause. The **finally** clause can be particularly useful for closing file handles, and freeing up other resources which might have been allocated at the beginning by a method.

6.10 Built in Exceptions in Java

The standard package **java.lang** defines several exception classes. The most general of these exceptions are subclasses of the standard type **RuntimeException**. **java.lang** is implicitly imported into all Java programs, therefore most of these exceptions are automatically available. These methods need not be included in any methods **throws** list either. Such exceptions are called as unchecked exceptions because the compiler does not check to see whether a method handles or throws these exceptions. Those exceptions which must be included in a method's **throws** list if the method can generate one of these exceptions and does not handle it itself.

Some Unchecked Runtime Exception Subclasses :

- ArithmeticException
- ArrayIndexOutOfBoundsException
- IllegalStateException
- NegativeArraySizeException
- IllegalMonitorStateException

Some Checked Exceptions defined in java.lang

- ClassNotFoundException
- IllegalAccessException

InterruptedException
NoSuchMethodException
Creating Exception Subclasses

In order to create your own exception types to handle situations specific to your application, define a subclass of **Exception**. (**Exception** is a subclass of **Throwable**). The **Exception** class does not define any methods of its own. It inherits those methods provided by **Throwable**. Thus all exceptions including those created by you have the methods defined by **Throwable** available to them.

The methods defined by **Throwable** include

Throwable fillInStackTrace()
String getLocalizedMessage()
String getMessage()
void printStackTrace()

6.9 & 6.10 Check Your Progress.

1. Write True or False.

- a) The finally block will be executed even if an exception is not thrown.
- b) The Exception class defines its own methods.
- c) java.lang has to be explicitly imported into all Java programs constructors.

6.11 SUMMARY

An exception in Java is an object that describes an exceptional i.e. error condition which has occurred in a piece of code. Exceptions can be generated by the Java run time system, or they can be manually generated by your code.

Exception handling in Java is managed by five keywords: try, catch, throw, throws, and finally. A try block contains the program statements which you want to monitor for exceptions. If an exception occurs within the try block, then it is thrown. Using catch, your code can catch this exception and handle it in the appropriate manner. The exceptions generated by the system are automatically thrown by the Java run time system. The keyword throw is used to manually throw an exception. Any exception that is thrown out of a method is specified by the throws clause. Any code that must absolutely be executed before a method returns is put in a finally block.

Source : www.elearning.shadoze.com(Link)

6.12 CHECK YOUR PROGRESS - ANSWERS

6.1 to 6.3

1. a) Throwable

- b) Exception, Error
 - c) throw
 - d) finally
2. a) An exception in Java is an object that describes an exceptional i.e. error condition which has occurred in a piece of code.
- b) Exceptions thrown by Java relate to fundamental errors which violate the rules of the Java language or the constraints of the Java execution environment.

6.4 to 6.6

1. a) True
- b) False
- c) True
- d) False

6.7 & 6.8

1. a) throw
- b) throws

2. a) False
- b) True

6.9 & 6.10

1. a) True b) False c) False

6.13 QUESTIONS FOR SELF - STUDY

1. What is an exception? List the five keywords used for exception handling in Java. Write and describe the general form of exception handling block.
2. Explain with an example how you will use the try and catch block.
3. Illustrate with example the nested try statement.
4. Describe the throw and throws statements using their general forms.
5. Write a short note on the keyword finally.

6.14 SUGGESTED READINGS

1. www.java.com
2. www.freejavaguide.com
3. www.java-made-easy.com
4. www.tutorialspoint.com
5. www.roseindia.net



This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.